

PYTHON PROGRAMMING:-

1st-SEM

BASICS OF PYTHON:-

- Python is a high level, interpreted, general-purpose programming language.
- Created by "Guido Van Rossum" in 1991.
- Python known for its simple syntax, readability and flexibility.
- Python widely used in web development, data science, artificial-intelligence, automation and more.

KEY FEATURES:-

- Easy to learn → Syntax is like English
- Interpreted → Runs code line by line
- Cross-platform → works on windows, Mac etc
- Object-Oriented → Support classes and objects
- Huge-libraries → Thousands of module for every domain.
- Open-Source → Free to use.

Comments:-

- Comments are ignored by the interpreter
- it is used for explanation/documentation.

Ex:-

This is a single line comment

→ '#' is used for single line comment

''' ''' → used for multiline comment

This is

a multiline comment

comment

''' '''

Identifiers:-

- :- Identifiers are names given to variables, functions, classes, modules, etc in python

Rules For Identifiers:-

- can only contain letters (A-Z, a-z), digit (0-9) and underscore.
Ex- name, students, max-value
- must not start with a zero →
 - 1) 1value - invalid
 - ✓ 2) value1 - valid
- case-sensitive →
 - 1) Python is a case-sensitive language.
 - 2) In Python 'A' is different than 'a'.
Ex: Name and name both are different identifiers.
- ✓ → special characters like (@, %, #) are not allowed.
- keywords cannot be used as identifiers.

KEY WORDS:-

- keyword are reserved words in python that have special meaning.
- They cannot be used as identifiers (variables, ^{names} function name, etc).
- Ex → if, else, while, for, break, continue, class, def
↳ define

Difference Betn Identifiers And Keywords:-

<u>Identifiers</u>	<u>Keywords</u>
→ Name given to variable, functions, etc.	→ Predefined reserved words in python
→ user defined	→ fixed already
→ Ex → age, student_name	→ Ex → if, class, for

Special Identifiers:-

- -single leading underscore - used for private variables (by convent)
- -double leading underscore - for name meaning in classes.

Variable:-

- Variable are fundamental in programming used to store data that can be referenced and manipulated in a program.
- ✓→ In simple words variable is a dataname where we can store different type of data value.

- 1) Introduction to variable
- 2) Declaring And assigning variable
- 3) Naming convention
- 4) Understanding variable types
- 5) Type checking and conversion
- 6) Dynamic Typing
- 7) Practical examples and common errors.

1. `a="xy"`
`type(a)`

- str → it represent string
→ string means a set of character

2. # Declaring And Assigning Variables.

`age=32`

`height=6.1`

`is_student=True`

→ Here we are assigning values to the variables

print the variables

`a=25`

`print("value of a is :", a)`

`print("age is :", age)`

`print("height:", height)`

→ print function is used to display

output:

value of a is : 25

age is : 32

height : 6.1

3. ## Naming Conventions

variable names should be descriptive

They must start with a letter or an '_' and contain numbers and underscore

Variable names are case sensitive

→ There are the rules for declaring a variable

first_name = "KRish" - valid variable Name

Last_name = "Harku"

01name = "OM" - } Invalid Variable Name

2-height = "6"

Case Sensitivity →

Name = "OM"

name = "om"

→ Here Both are different, Because Python is case-sensitive means → (A ≠ a)

Types of variables →

age = 25 → Integer (int)

Height = 6.1 → float (float)

name = "OM" → string (str)

is_student = True → Boolean (bool)

4. age1 = 25
age2 = "25"
print(type(age1))
print(type(age2))

output →

<class 'int'>

<class 'str'>

NOTE: type() is a built-in function which determine which type of a variable.

5. ## Dynamic Typing

Python allows the type of a variable to change as the program executes

var = 10 # int

print(var, type(var))

var = "Hello"

print(var, type(var))

var = 3.14

print(var, type(var))

output: → 10 <class 'int'>
Hello <class 'str'>
3.14 <class 'float'>

DATATYPES:-

Defination →

- Data types are classification of data which tell the compiler or interpreter, how the programmer intends to use the data.
- They determine the type of operation that can be performed on the data, the value that the data can take, and the amount of memory needed to store the data.

Importance:

- Data types ensures that data is stored in an efficient way.
- They help in performing correct operation on data.
- proper use of data types can prevent errors and bugs in the program.

→ Basic Data types → Integer, Floating-point, Strings, Boolean
→ Advanced Data types → List, Tuples, Sets, Dictionaries.

Syntax of DataTypes:-

ex → `text+1 = "Hello"`
`text+2 = "World"`
`print(type(text+1))`

output :-

`<class 'str'>`

ex → `#integer`
`x = 10`
`print(type(x))`

`#float`
`y = 20.5`
`print(type(y))`

output:-

`<class 'int'>`

`<class 'float'>`

Operator:-

Defn →

Operator is a special symbol that is used to perform certain operations
Ex → +, -, *, /, ==, and, etc.

Types Of Operator :-

1. Arithmetic Operator :-

- it is used for mathematical operation.
- Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%) , Exponent (**) , floor division (//)
 gmp it means power
- These symbols are the 'Arithmetic Operator' which are used in various mathematical operation.
- floor division operator (//) → it returns only integer part
Ex → $10 // 3 = 3$
- modulus → it is used to find remainder and it only return the remainder.
Ex → $4 \% 3$ → here remainder is 1 so it returns only one.

code → # Arithmetic Operator

```
a, b = 10, 3
print(a+b) # 13
print(a-b) # 7
print(a*b) # 30
print(a/b) # 3.33...
print(a%b) # 1
print(a**b) # 1000
print(a//b) # 3.
```

2. Relational (comparison) Operator :-

✓ → used to compare two values. Always return True or False

operator	Meaning	Example	Output
==	equal to	$5 = 5$	True
!=	Not equal to	$5 != 3$	True
>	greater than	$5 > 3$	True
<	less than	$5 < 3$	False
>=	greater than or equal to	$5 >= 2$	True
<=	less than or equal	$2 <= 2$	True

code →

```
x, y = 10, 20
print(x == y) # False
print(x != y) # True
print(x > y) # False
print(x < y) # True
```

3. Logical Operators :-

→ it is used to combine conditional statements.

Operator **Meaning**

and

True if both condⁿ are true

or

True if at least one condⁿ is true

not

Reverse the result

ex →

Example

(5 > 3 and 10 < 5)

not (5 > 3)

output

False

True

code →

```
a, b = True, False
print(a and b) # False
print(a or b) # True
print(not a) # False
```

4. Assignment Operator :-

→ Used to assign values to variables.

Operator → =, +=, -=, *=

ex → x = 5, x += 3 means x = x + 3

x -= 3 means x = x - 3

Operator Precedence →

→ precedence means which operator is evaluated first when an expression has more than one operator.

→ it states that the operator which will evaluate first has higher precedence.

ex → result = 10 + 5 * 2

print(result) # 20 (not 30)

→ Here * has higher precedence than +.

Operator Associativity →

- if two operator have same precedence then associativity decides the evaluation order.
- According to Operator Associativity evaluation is done from Left to Right

ex → `print(100/10*2)`

first $(100/10) = 10$, then $10 * 2 = 20$

NOTE →

- Some operator like exponent $(**)$, assignment $=$, $+=$ are done from Right to Left

ex → `print(2**3**2)`

first $3**2 = 9$ then $2**9 = 512$

Operator Precedence Table :-

No of precedence

operator

1

`()`

2

`**`

3

`+, -, *`

4

`*, /, //, %`

5

`+, -`

6

`<<, >>`

7

`&` ← Bitwise And

8

`^` ← Bitwise XOR

STRING:-

→ A string is a set of characters enclosed in single quotes ("), double quotes (" ") or triple quotes ("''" or ("''' '"))

Ex →
s1 = 'Hello'
s2 = "World"
s3 = '''Python
is powerful'''

NOTE → triple quotes are used for multiline string.

Accessing Characters (Indexing):-

→ Python uses zero-based indexing.
→ Negative indexing starts from the end.

Ex → text = "python"
print(text[0]) # p
print(text[-1]) # n

Easy ex → Name = "manthan"

→ from left to right index starts from zero (0).

gmp so here manthan
0 1 2 3 4 5 6

→ From Right to left index starts from (-1).

gmp so here - manthan
-7 -6 -5 -4 -3 -2 -1

String Slicing:-

We can extract parts of strings using slicing.

Ex → s = "Python"
print(s[0:4]) # Pyth
print(s[0:]) # Python
print(s[:3]) # Pyt
print(s[2:]) # thon
print(s[::-1]) # nohtyp (reverse string)

N.T → s[0:4] → from left it will consider 0 index but from right it will consider (-1) index from that index. 0 6 5 4 3 2 1
left side 6 5 4 3 2 1
right side 0 1 2 3 4 5 6
show 0 1 2 3 4 5 6

gmp → if we try to slice the string from Right to left it will show value error

Ex → s = "python"
print(s[4:0]) → it is invalid
print(s[-1:-4])

STRING OPERATION:-

1. Concatenation (+)

Ex → `print("Hello" + "World")` # HelloWorld

2. Repetition (*)

Ex → `print("Hi!" * 3)` # Hi! Hi! Hi!

3. Membership (in/not in)

→ This operator is used to check wheather a value (char, substring or element) exists in a sequence.

→ in - Return True if the value is present

→ not in - Return False if the value is ~~not~~ present.

Ex → `text = "Python Programming"`
`print("Python in text")` # True
`print("Java in text")` # False
`print("c++ not in text")` # True

4. len() -

→ The len() function in python is used to find the total number of items in a sequence (str, list, tuple etc).

Ex → `name = "Python"`
`print(len(name))` # 6

Syntax: `len ( )` → variable name

Escape characters

\n - Newline

\t - Tab

\\ - Backslash

\' - Single Quote

\\" - Double quote

String Immutability -

- strings are immutable - cannot be changed after creation.
- if you want to change the value of a string then you have to create a new string.

Ex →
s = "Hello"
s = s + "world"
print(s) # Hello world.

OTHER STRING METHOD:-

- lower() - converts string to lowercase
- upper() - converts string to uppercase
- title() - capitalizes first letter of all words
- capitalize() - capitalizes first letter only
- strip() - Remove spaces (left & right)
- lstrip() - Remove spaces (from left)
- rstrip() - Remove from Right
- find() - substring (Return index of substring - 1 if not found)
- replace() - Replace substring
- index() - same as find but error if not found
- count() - Count occurrences of substring

Q: Difference Betⁿ Mutable & Immutable

Mutable

Defn - object whose value can be changed after creation

- when you modified them the same object in memory get updated (no new objects)
- Ex - list, set

Im-mutable

Defn - object whose value cannot be changed after creation.

- if you try to change them a new object is created in memory
- Ex - String, tuple, int, float

LIST

Definition →

- A list in python is a mutable, ordered collection of items.
- it can store different data types (int, str, float).
- Lists are written inside square brackets [].

Example →

```
my-list = [10, "hello", 3.14, True]
print(my-list) # [10, 'hello', 3.14, True]
```

Properties Of List

- 1- Order - elements keep their position
- 2- Mutable - elements can be changed
- 3- Heterogeneous - can hold diff. data types
- 4- Allow duplicates - same element can appear multiple times.

Creating list

empty = [] → A list with zero element is called as empty list
number = [1, 2, 3, 4, 5, 6]

mixed = [1, "python", 3.5, True]

Accessing Elements:-

1. Indexing → list[index] start from 0 → left to Right

2. Negative indexing → starts from -1 → Right to left

Ex → fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # apple
print(fruits[-1]) # cherry

Slicing:-

```
num = [10, 20, 30, 40, 50]
print(num[1:4]) # [20, 30, 40]
print(num[:3]) # [10, 20, 30]
print(num[2:]) # [30, 40, 50]
print(num[0:2]) # [10, 20]
print(num[::-1]) # [50, 40, 30, 20, 10]
```

✓ gmp
↳ step size can not be 0

NOTE → num[:3]

→ it's called a step

→ step size zero thiba heta ta element skip ho

List Operation:-

1. Concatenation (+)

→ `a = [1, 2]`
`b = [3, 4]`
`print(a+b)` # `[1, 2, 3, 4]`

2. Repetition (*)

→ `print([0]*4)` # `[0, 0, 0, 0]`

3. Membership (in, not in)

→ `print(3 in [1, 2, 3])` # `True`
`print(5 not in [1, 2, 3])` # `True`

4. Length (len())

→ it is used to determine the length of a list
mean - how many elements are there in a list.

→ `print(len([1, 2, 3, 4]))` # `4`

List Methods:-

→ `append(x)` - Add element at the end of the list

Ex → `num = [5, 3, 8, 1]`

`num.append(10)`

Output - `[5, 3, 8, 1, 10]`

→ `insert()` - Using this function we can add element at a particular index.

Ex → `num = [5, 3, 8, 1]`

`num.insert(1, 10)`

Output - `[5, 10, 3, 8, 1]`

→ `remove()` → Removes first occurrence of value

→ `pop()` - Removes and return element at index
(By default last)

→ `index()` - Return index of first occurrence

→ `clear()` - Remove all elements

→ `count()` - counts occurrence of element

→ `reverse()` - Reverses list order.

→ `copy()` - Returns a shallow copy

→ `sort()` - Arrange the list in Ascending Order.

✓ Difference Between List and String.

List

Defn - A collection of multiple elements enclosed in square bracket.

- Mutable (sequence)
- stores different data type
- used to store and manipulate collections of data
- Ex - `lst = [40, "apple", 3.5]`
- We can directly change list element

String

Defn - String is a set of character enclosed in single (' ') or double (" ") quotes

- Immutable sequence
- can store only character
- used to handle and manipulate text data
- Ex - `str = "Hello"` directly
- we can't change characters.

Tuple

Defn -

or

→ A tuple is a collection of datatypes in Python

→ It is used to store multiple items in a single variable

→ Tuples are similar to list, but tuples are immutable

↳ What makes tuples different from List.

→ Items are ordered, allow duplicates, and can be of different datatype.

→ tuple elements are enclosed in parenthesis ().

Ex →

```
my_tuple = (10, "Hello", 3.14, True)
print(my_tuple)
```

output:

```
(10, 'Hello', 3.14, True)
```


Tuple Characteristics:

1. Ordered - Items have a fixed order
2. Immutable - Once created elements cannot be changed (directly).
3. Heterogeneous - can store int, float, string, list
4. Indexed - Elements can be accessed using index.
5. Allows duplicate values - can store same value multiple times

CREATING TUPLE:

① `t = (1, 2, 3)`

② `t = ()`

↳ it is called as empty tuple

A tuple with zero elements is called 'Empty Tuple'.

Accessing Tuple Elements:-

→ We can access tuple elements by their index number.

Ex:- `t = (10, 20, 30, 40, 50)`

`print(t[0])` # 10

`print(t[-1])` # 50

`print(t[1:4])` # (20, 30, 40)

Tuple Operation:-

concatenation - Join two tuples

`(1, 2) * (3, 4)` # (1, 2, 3, 4)

Repetition - Repeats elements

`(1, 2) * 2` # (1, 2, 1, 2)

membership - check if item exists

`3 in (1, 2, 3)` # True

Length

- count elements

`len((1, 2, 3))` # 3

Max value

- Returns max value

`max((1, 2, 3))` # 3

Min value

- Returns min value

`min((1, 2, 3))` # 1

Sum

- Add all numeric item

`sum((1, 2, 3))` # 7

Indexing

- Access element by index no

`t[1]` # 2

Slicing

- Range of elements

`t[0:2]` # 1, 2

Tuple Methods (Only 2 Built-in Methods)

→ `count()` syntax `t.count(x)` → count how many times 'x' appears

Ex → `@1,2,3).count(1) → 2`

→ `index()` syntax `t.index(x)` → Returns index of first occurrence of 'x'

Ex → `(10,20,30).index(20) → 1`

Tuple Functions :-

`len()` → Total elements

`max()` - Maximum value

`min()` - Minimum value

`sum()` - Sum of elements

`sorted()` - Return sorted list

`any()` - True if any statement is true

`all()` - True if all statement is true

`tuple(iterable)` - converts iterable into tuple

Type Conversion :-

List → Tuple

`list1 = [1,2,3]`

`t = tuple(list1)`

`print(t) # (1,2,3)`

Tuple → List

`t = (1,2,3)`

`list1 = list(t)`

`print(list1) # [1,2,3]`

✓ Tuple vs List

Feature	Tuple	List
Syntax	<code>()</code>	<code>[]</code>
Mutable	NO	Yes
speed	faster	slower
Methods	Only 2	Many
use	Fixed data	changeable data
Memory	Less	More

Defn → A unique

Key Po

- No du
- Un or
- muta
- ves

Ex - my. pro

CRG

S1:

S2:

S3:

9m

- N

-

-

-

SET

Defn → A set is an unordered, unindexed and mutable collection of unique elements

Key Points:-

- No duplicate items allowed
- Unordered (no fixed position)
- Mutable
- Uses $\{ \}$ - curly braces

Ex -

my-set = {10, 20, 30, 40}

print(my-set) # {10, 20, 30, 40} Order may vary

CREATING SETS:

S1 = {1, 2, 3, 4}

S2 = {1, 1, 2, 3, 4}

S3 = {1, 5, 6, 7}

→ Duplicates not allowed

Important Rules:

- No duplicates elements
- unordered
- Elements must be immutable types (int, string, tuple)

Ex → S = {1, "hello", (2, 3)} # valid

S = {1, [1, 2], 3} # invalid (list not allowed)

Accessing Set Elements:

- We can't access elements using index
- But we can loop through it

Ex - for item in {10, 20, 30}:
print(item)

Set Operation:

<u>Operation</u>	<u>Symbol/function</u>	<u>Meaning</u>
Union		- All elements of both sets
Intersection	& or <code>set1.intersection(set2)</code>	- Common elements
Difference	- or <code>set1.difference(set2)</code>	- only in first set
Sym. Difference $(A-B) \cup (B-A)$	Δ or <code>set1.sym_diff(set2)</code>	- elements not common
Subset	<code>set1 <= set2</code> or <code>issubset()</code>	- All elements of S_1 in S_2
Superset	<code>set1 >= set2</code> or <code>issuperset()</code>	- All elements of S_2 in S_1
Disjoint	<code>isdisjoint()</code>	- No common element

SET METHODS :-

<u>METHOD</u>	<u>MEANING</u>
<code>add()</code>	- Add element to the set
<code>update()</code>	- Adds multiple elements
<code>remove()</code>	- Remove a specific element (error if not found)
<code>discard()</code>	- Remove a specific element (No error if not found)
<code>pop()</code>	- Remove a random element from the set
<code>clear()</code>	- make set empty
<code>copy()</code>	- Returns a copy of the set
<code>union()</code>	- combine elements of 2 sets

SET-BUILT IN FUNCTIONS

<u>Function</u>	<u>DESCRIPTION</u>
<code>len()</code>	- Total no. of elements
<code>max()</code>	- Maximum element
<code>min()</code>	- Minimum element
<code>sum()</code>	- sum of elements
<code>sorted()</code>	- Returns sorted List
<code>any()</code>	- True if any element True
<code>all()</code>	- True if all element True

FROZEN SET (IMMUTABLE SET)

- A frozen set is a set that cannot be changed (immutable)

ex - `f = frozenset([1, 2, 3])`
`print(f)`
`#f.add(4) X ERROR (can't modify)`

gmp

Difference Between Set, Tuple, List

→

<u>FEATURE</u>	<u>SET</u>	<u>LIST</u>	<u>TUPLE</u>
Brackets	{ }	[]	()
Ordered	No	Yes	Yes
Index	No	Yes	Yes
Mutable	Yes	Yes	No
Duplicates	No	Yes	Yes
Speed	Fastest	Medium	Fast

ADVANTAGES AND DISADVANTAGES OF SET

ADVANTAGES :-

- Remove duplicates automatically
- Fast Searching and comparison
- Mathematical operations
- Can be easily converted into List

DIS-ADVANTAGES :-

- Unordered
- Cannot store mutable items
- Not useful if duplicates are required

Conditional Statement

→ Conditional statements are used to make decisions in a program. They allow you to execute certain blocks of code depending on whether a condition is 'true' or 'false'.

1. if statement:-

The simplest conditional.

if the condition is true the block runs, otherwise it is skipped.

Ex → age = 18
if age >= 18:
 print("You are eligible to vote.")

2. if-else statement:-

→ If there are two conditions we can use if-else statement
→ syntax

9mp
if condition:
 statement
else:
 statement

if the condition is true
this block runs
→ else it will run the else block

This curly braces are not used in python

Ex → num = 10;
if num % 2 == 0:
 print(num, "is an even number.")
else:
 print(num, "is an odd number.")

3. if-elif-else statement:-

→ if there are multiple statements we can use if-elif-else statement.

→ if 1st condition is true it will run 1st statement and comes out of the block
otherwise, it will check 2nd condition if it is true it will run the 2nd elif block otherwise it will directly run else part

syntax

```
if condition:
    print("statement")
elif condition:
    print("statement")
elif condition:
    print("statement")
else:
    print("statement")
```

> first this execute
✓ then elif, then if all the condition are not satisfied then else: statement will execute.

3. Nested if else

- A nested if-else means having an if statement inside another if (or else) block.
- It is used when we need to check multiple condition in a hierarchical (step-by-step) way.

Syntax:-

```
if condition1:
    # executes if condn 1 is True
    if condition2:
        # executes if condn 2 is also True
    else:
        # executes if condn 1 is True but condn 2 is not True
else:
    # executes if condition 1 is False
```


LOOP

→ Defn:-

A loop is used to execute a block of code repeatedly until a certain condition is met.

→ Python has mainly 2 types of loops:

1- for Loop

2- while Loop

- FOR LOOP:-

syntax:-

for variable in sequence:

code blocks

Explanation :-

- Used to iterate over a sequence (like list, tuple etc)

- Runs automatically for each items in the sequences.

ex-

```
for i in range(5):
```

```
    print(i) # 0
```

```
            1
```

```
            2
```

```
            3
```

```
            4
```

WHILE - LOOP :-

→ While loop executes a block of code as long as the condition is true

syntax :-

while condition:

body of loop

ex:

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i = i + 1
```

Explanation :-

- The loop continues until the condition $i <= 5$ becomes false. Once i becomes 6, the condition fails and loop stops.

Statement

break

Description

- Immediately stops the loop

Example

- stops when a condition is met

continue

- Skips the current iteration and continues with the next one

- Used to skip specific values

pass

- Does nothing (a placeholder statement)

- used when you don't want to execute any code

Example of Break:-

code → for i in range(5):
 if i == 3:
 break
 print(i)

output:
0
1
2

Example of Continue:-

code- for i in range(5)
 if i == 2:
 continue
 print(i)

Output: 0
1
3
4

Example of pass:-

code- for i in range(5):
 pass
 print("Loop executed")

'ELSE' with LOOPS :-

→ Python Provides a unique feature : Loops can have an else block.

The else part is executed only when the loop completes normally (i.e. no break is used)

FOR LOOP WITH ELSE:-

→ for i in range(3):

print(i)

else:

print("Loop finished successfully")

output

0

1

2

Loop finished successfully

WHILE LOOP WITH ELSE:-

i = 0

while i < 3:

print(i)

i += 1

else:

print("while loop ended")

output

0

1

2

while loop ended

NOTE

If a loop exists using break, the else part will not be executed

Nested Loops :-

- When one loop is placed inside another loop it is called as nested loop.

```
- for i in range(3):  
    for j in range(2):  
        print(i,j)
```

Output

0 0
0 1
1 0
1 1
2 0
2 1

INFINITE LOOP :-

A loop that never ends because its condition is always true is called an infinite loop

- while True:

```
    print("This is an infinite loop")
```

LOOP WITH RANGE FUNCTION :-

The range() function is often used with loops

Syntax -

```
range(start, stop, step)
```

Ex -

```
for i in range(1,10,2):  
    print(i)
```

Output

1
3
5
7
9

8. Difference Between for loop and while loop.

FOR-LOOP

- It is used to iterate over a sequence (like list, tuple, string etc)
- used when the number of iteration is known in advance

syn for variable in sequence

ex for i in range(5):
 print(i)

WHILE-LOOP

- It is used to execute a block of code repeatedly as long as a given condition is true.
- Used when no. of iteration is unknown and depends on a condition

syn - while condition:

ex - i=0;
while i<5;
 print(i);
 i+=1

Dictionary

- A dictionary is an unordered, mutable, indexed collection of key value pairs.

- syntax:-

^{gmp} $d = \{key: value, key: value, key: value\}$

- Each key must ~~contain~~ be unique and immutable.

- values can be of any type and can be repeat.

CREATING DICTIONARIES :-

$d = \{ 'name': 'OM', 'age': '19' \}$ // method 1 using $\{ \}$

$d = \text{dict}(\text{name}='OM', \text{age}='19')$
 $\swarrow$ key $\searrow$ value

ACCESSING ELEMENTS :-

<u>Access Type</u>	<u>Code</u>	<u>Description</u>
Direct Access	$d['name']$	- Raises key error if not found
Safe Access	$d.get('name')$	- Returns None if not found
With default	$d.get('city', 'Not found')$	- Avoid error with default value

ADDING & MODIFYING

$d['age'] = 25$ # modify

$d['country'] = 'India'$ # add or modify

$d.update(\{ 'city': 'Mumbai' \})$ # update

DELETING ELEMENTS

<u>Method</u>	<u>Example</u>	<u>Effect</u>
$\text{del } d[key]$	$\text{del } d['age']$	- Removes specific key
$\text{pop}()$	$d.pop('key')$	- Removes + return value
$\text{popitem}()$	$d.popitem()$	- Removes last inserted
$\text{clear}()$	$d.clear()$	- Removes all elements
$\text{del } d$	$\text{del } d$	- Deletes entire dictionary

Dictionary Methods :-

Method	Description	Example
Keys()	- Returns all keys view	- d.keys()
Values()	- Return all value view	- d.values()
Items()	- Return key-value pairs	- d.items()
Copy()	- Creates shallow copy	- d2 = d.copy()
update()	- Add or overwrites pairs	- d.update(d2)
set default(k, v)	- Returns value of key exist, else insert	- d.setdefault('z', 0)
fromkeys(seq, val)	- Creates dict with same value	- dict.fromkeys('abc', 1)

LOOPING THROUGH DICTIONARY :-

```
for key in d:  
    print(key)
```

```
for key, value in d.items():  
    print(key, value)
```

```
for value in d.values():  
    print(value)
```

✓ NOTE:-

• keys(), values(), items() return dynamic view objects. automatically reflect any dictionary changes.

Dictionary Comprehensions :

Syntax:-

{key: expr: value: expr for item in iterable if condition}

Example:-

squares = {x: x**2 for x in range(5)}

even-squares = {x: x**2 for x in range(6) if x%2 == 0}

swap = {v:k for k,v in {'a':1, 'b':2}}

UNIT-1

INTRODUCTION TO COMPUTERS

Defn:- A computer is an electronic device that accept data as input, processes it according to a set of instructions, program stores data/result and produces information as output.

CHARACTERISTICS OF COMPUTER :-

- 1- **Speed** - perform millions of operations per second
- 2- **Accuracy** - Gives correct result if instructions are correct
- 3- **Diligence** - Not tired
- 4- **Storage** - store huge amount of data
- 5- **Automation** - works automatically once programmed
- 6- **Versatility** - perform different type of tasks.

LIMITATION OF COMPUTERS :-

- 1- No intelligence or emotion
- 2- Can't think or take decision on its own
- 3- Works only as per instructions

GENERATIONS OF COMPUTER :-

FIRST GENERATION (1940-1956)

- Technology - Vacuum Tube
- Machine Language
- Ex:- ENIAC, EDVAC
- Very large, high power consumption.
- Disadvantages:- overheating, costly, unreliable

SECOND GENERATION (1956-1963)

- Technology - Transistors
- Language - Assembly language
- Faster and smaller than 1st Gen
- Example : IBM 7094
- Less heat generation.

- 1944-1971)
- Technology :- Transistors Integrated Circuits (ICs)
 - Language :- High-level languages (fortran, cobol)
 - More reliable and efficient
 - Ex:- IBM system/360

4th Generation (1971-Present)

- Technology :- Microprocessor
- Personal computers introduced
- Example :- Intel 8086, IBMPC
- Use of operating systems

Fifth Generation (Present & future)

- Technology : AI, ML
- Example :- Robotics, Experts systems
- focus on intelligent computing

Types Of Computers :-

i/ Based on size and performance :-

i/ Super Computers :-

- High processing power
- Used in weather forecasting, space research
- Example : PARAM, CRAY

ii/ Mainframe Computers

- Used by banks, railways
- Handle thousands of users

iii/ - Minicomputers

- Medium - sized - systems
- Used in small organizations

Micro-computers

- Personal computers
- Desktop, Laptop, tablet, smartphone

Based on purpose

- 1- General purpose computers - perform multiple tasks
- 2- Special purpose computers - Designed for specific tasks

COMPUTER SYSTEM HARDWARE :-

Block Diagram of computer system

- 1- Input unit
- 2- CPU (Central Processing Unit)
- 3- Memory unit
- 4- Output Unit

CENTRAL-PROCESSING-UNIT (CPU)

- It is also called as brain of computer

Components of CPU -

1- Arithmetic Logic Unit (ALU)

- performs arithmetic operations (+, -, $\times$, $\div$)
- performs logical operations (AND, OR, NOT)

2- Control Unit (CU)

- controls and coordinates all activities
- Directs data flow between memory and ALU

3- Registers

- High-speed memory inside CPU
- store instructions and data temporarily

COMPUTER MEMORY UNIT

1. Memory Unit (overview block)

The memory unit is a part of the computer systems that stores

- Data
- instruction (programs)
- intermediate results
- final results

- It provides information to the CPU whenever required.

Functions of Memory Unit

- stores input data before processing
- stores instructions during execution
- stores results temporarily or permanently
- supplies data to CPU at high speed

2. Memory Hierarchy Block :-

Memory is organized in a hierarchy based on speed, cost and storage capacity

Memory Hierarchy order :-

- 1- CPU registers
- 2- Cache Memory
- 3- Primary Memory
- 4- Secondary memory

As we go down :- $\rightarrow$ speed $\downarrow$, cost $\downarrow$, storage capacity $\downarrow$

3. CPU - Registers Block :-

Registers are small, high-speed memory locations inside the CPU

Characteristics :-

- fastest memory
- very small storage
- Expensive
- volatile

- Accumulator
- Program Counter (PC)
- Instruction Register (IR)
- Memory Address Register (MAR)
- Memory data Register (MDR)

v- High speed
Direct Access by
CPU

CACHE MEMORY BLOCK

Defn:- Cache Memory is a high-speed memory placed between CPU and main memory.

Purpose :-

Reduces access time
stores frequently used instructions data

CHARACTERISTICS :-

- faster than RAM
- slower than registers
- volatile
- small capacity

Types Of Cache :-

1. L1 cache - Inside CPU, fastest
2. L2 cache - slightly slower
3. L3 cache - shared cache

ADVANTAGES

- Improves CPU performance
- Reduces processing time

DISADVANTAGES :-

- Expensive
- limited size

MAIN MEMORY / PRIMARY MEMORY BLOCK

Defn:- Primary memory is the main working memory of the computer.

CHARACTERISTICS :-

Directly Accessed by CPU
Faster than secondary memory
volatile/non, volatile
Limited storage

Types :-

- 1- RAM
- 2- ROM

RAM

RAM

used

- Vol

- R

- T

- T

Ty

1.

2.

✓

✓

Dis

RAM (Random Access Memory)

RAM stores data and instructions that are currently being used.

- Volatile
- Read and Write memory
- Fastest Access

Types of RAM

1. SRAM (Static RAM) - faster, costly, used in cache
2. DRAM (Dynamic) - slower than SRAM, cheaper, used in main m-

✓ ADVANTAGES :- High speed, Direct CPU Access.
DIS- ADV:- Data lost when power is off.

ROM (Read-Only-Memory)

ROM stores permanent instructions required for booting.

- Non-volatile
- Read only
- Stores, firmware

PROM - Programmable once

EPROM - Erasable using UV light

EEPROM - Electrically erasable

✓ ADV - permanent storage, Reliable

DIS- ADV - can't be easily modified

SECONDARY MEMORY BLOCK :-

Secondary memory is used for permanent storage of data and programs

Types

- non-volatile
 - Large storage
 - slower than main
 - cost effective
- 1- Magnetic storage :- Hard disk, Magnetic tape
 - 2- Optical storage :- CD, DVD, Blu-ray
 - 3- Solid state storage :- SSD, Pen drive, Memory card

✓ ADV - Permanent storage, Large capacity

DIS ADV :- slower than RAM, Not directly accessed by CPU

VOLATILE

Data loss

Data loss when power off

Ex:- RAM

NON-VOLATILE

No - data loss

No data loss when power off

Ex:- ROM, HDD

COMPUTER NETWORKING

Defⁿ:- A computer network is a system in which computers are connected to share information, and resources. This means that the computers can talk to each other and that every computer in the network can send information to the others.

NEED OF NETWORK :-

- To share data at different locations of the organization.
- Share information with other people on the network.
- Make quick link with the other person using network.
- Communicate in the form of mail messages and chatting.
- Gain knowledge from the worldwide resources using internet.

ELEMENTS OF A NETWORK :-

HOST :-

Main or controlling computer connected to other computers or terminal to which it provides data or computing services via a network.

SENDER (Source Node)

The sender is the device that initiates communication and sends data over the network.

Examples :-

- Computer
- Mobile phone
- Server

Functions :-

- creates data
- Converts data into signals
- sends data to receiver.

2. Receiver (Destination Node)

The receiver is the device that receives data sent by the sender.

Examples :-

- Computer
- printer
- smartphone

Functions :-

- Receives signals
- converts signal back into data.
- Uses the received information.

TYPES OF COMPUTER NETWORK :-

A computer network can be classified into different types based on,

- Geographical area covered
- size
- purpose.

The most common types are :-

LAN
MAN
WAN

1. LAN - LOCAL AREA NETWORK

A LAN is a network that connects computers and devices within a small geographical area, such as room, building, school or office.

- covers small area
- High data speed
- Low cost
- privately owned
- High reliability

- Examples
- school computer lab
 - office network
 - Home wi-fi network

ADVANTAGES

- Easy setup
- High security
- High speed
- Resource sharing (printers, files)

DISADVANTAGES

- Limited Area coverage
- Initial setup cost
- Requires skilled administration

2. MAN - Metropolitan Area Network :-

A metropolitan area network connects multiple LANs within a city or metropolitan area.

- covers larger area
- Medium speed
- can be public or private
- Higher cost than LAN
- Used in metro cities
- Range 5-50 km

3. WAN - Wide Area Network :-

- WAN stands for Wide Area Network
- Range above 50 km
- High speed communication
- costly to setup and maintain

4. PAN - Personal Area Network

- PAN stands for Personal Area Network
- High speed
- Range is short upto 10 km
- Ex. - data sharing between a few devices

'NETWORK' - BASED ON TYPES OF COMMUNICATION

- 1- Point-to-Point Network
- 2- Multipoint Network
- 3- Broadcast Network

BASED ON ARCHITECTURE

- P2P (Peer to Peer)
- Networks
- Client server network
- Hybrid Network.

NETWORK DEVICES :-

Defn:- Network Devices are hardware components used to connect computers and other devices in a network and to control, manage and transmit data efficiently from one device to another.

- ✓ They play a crucial role in ensuring reliable communication, data transfer and network security.

1. REPEATER

It is a network device that receives weak signals, regenerates them and retransmits the signals to extend the network distance.

Functions:-

- Amplifies weak signals
- Extends Network coverage

Characteristics :-

- Works at physical Layer
- Does not filter data.

Advantages

- Improves signal strength
- Increases transmission distance

A 'HUB' is a basic networking device that connects multiple computers and sends data to all connected devices.

Functions

- Acts as a central connection point
- Broadcast data to all ports

Characteristics

- Works at physical layer
- No filtering or intelligence
- Half-duplex communication

Advantages :-

- Easy to install
- Low cost

3. Switch :-

A switch is an intelligent network device that connects multiple device and forwards data only to the intended destination using 'MAC' Address.

Function :-

- Reduces network traffic
- Improves performance

Characteristics

- Works at Data-Link-Layer
- Support full duplex communication
- Faster than 'HUB'.

4. BRIDGE :-

It is used to connects two similar LANs and filter network traffic.

FUNCTIONS

- Traffic filtering
- Network segmentation

CHARACTERISTICS :-

- Works at Data Link Layer
- Uses MAC Address

5. ROUTER

A Router connects different networks and determines the best path for data transmission using IP addresses.

Functions

- Routing data packets
- Connecting LAN to WAN
- Provides Internet Access

Characteristics

Works at Network Layer
Uses routing table
Intelligent device

6. GATEWAY

A Gateway connects networks using different protocols

Functions

Protocol conversion
Data translation

Characteristics

Works at all layers
Most intelligent device

7. MODEM (Modulator - Demodulator)

- It converts digital data into analog signals and vice-versa

Functions

Enables internet connectivity
Signal conversion

DEVICE	LAYER	FUNCTION
Repeater	Physical	Signal regeneration
Hub	Physical	Broadcast data
Switch	Data link	Forward data
Bridge	"	connect LANs
Router	Network	Route packets
Gateway	All layers	Protocol conversion

COMPUTER SOFTWARE

Defn:-

Computer software is a collection of programs procedure and related documentation that tells the computer what to do and how to do it. Software is intangible (can't be touched)

- Software acts as an interface between the user and the computer hardware. Without software hardware is useless.

KEY POINTS :-

- Software is a set of instructions
- It controls the operation of hardware
- It helps users perform specific tasks
- Software is written in programming languages

EXAMPLES :-

Window, Linux

Ms word, Excel

Antivirus programs

Types Of Software :-

1. System Software
2. Application Software
3. Utility Software

1. SYSTEM SOFTWARE :-

- It manages and controls the computer hardware and provides a platform for running application software.
- It acts as a bridge between hardware and application software.

Functions Of System Software :-

- Controls and manages hardware resources
- Manages memory, processor, and input/output devices
- Provides user interface
- Runs application software

Types of System Software :-

1. Operating System
2. Device Drivers
3. Language translator
4. Firmware

2. APPLICATION SOFTWARE :-

- It is designed to perform specific tasks for the user.
- It runs on top of system software and helps users solve real-life problems.

Features:

- User oriented
- Easy to use
- Task specific

Examples

Ms Office - General purpose

- used for common tasks.

Banking software

Railway reservation system
etc.

Developed for a specific organization or task.

3. UTILITY SOFTWARE :-

Utility software is a type of system-related software designed to maintain, manage and optimize computer performance.

Functions :-

- Protect system from viruses
- Improves system performance
- Manages files and disk space
- Detects and fixes errors

Examples :-

- Antivirus software
- Disk cleanup
- Disk Defragmenter
- Backup software

FIRMWARE AND THEIR USES :-

Firmware Defn:-

Firmware is a type of system software that is permanently stored in non-volatile memory such as ROM, EEPROM or Flash memory.

- It provides low-level control for a device's specific hardware.

- Firmware lies between hardware and software and helps the hardware to function properly.

CHARACTERISTICS OF FIRMWARE :-

Stored in ROM/Flash

Non-volatile

Provides basic control instructions

Device-specific

Faster than Application Software

Rarely modified, but can be updated.

TYPES OF FIRMWARE:-

Bios Firmware

Embedded Firmware

UEFI Firmware

Peripheral Device Firmware

EXAMPLES OF FIRMWARE :-

Bios in computers

Firmware in printers and scanners

Firmware in routers and modems

Firmware in mobile phones

Firmware in TV's machines.

USES OF FIRMWARE :-

Bootling the computers

Hardware control

Device communication

Embedded system operation

Performance and security improvement

Power Management

Difference Between Firmware And Software

FIRMWARE

Stored in ROM/Flash
Device-specific
Low-level control
Rarely changed
Ex - BIOS

SOFTWARE

Stored in Hard Disk
User-oriented
High-level tasks
Always updated
Ex - Microsoft

1. Program Development :-

Program development is a systematic process of designing, writing, testing and maintaining a computer program to solve a specific problem.

Steps Of Program Development :-

- Problem Analysis
- Program Design
- Program Development (coding)
- Testing and Debugging
- Documentation and Maintenance

1. PROBLEM ANALYSIS :-

Problem Analysis is the first step of program development in which the problem is clearly understood.

KEY POINTS :-

- Identify the input, process and output
- Understand constraints and requirement
- Break the problem into smaller parts

PROGRAM DESIGN

It is the process of planning the logic and structure of a program before writing the actual code.

Tools Used In Program Design :-

- Algorithm
- Pseudocode
- Flow chart